

Widoki, widoki zmaterializowane,
dynamiczne tabele

Dlaczego to nie ten sam problem

- Klasyczne pytanie „co jest szybsze?” jest źle postawione.
 - View daje wirtualizację zapytania,
 - Materialized view przechowuje precomputed result dla szybszego odczytu,
 - Dynamic table materializuje wynik definicji i utrzymuje go automatycznie według polityki odświeżania

Widoki

- CREATE OR ALTER VIEW
 - Tworzy nowy widok, jeśli jeszcze nie istnieje, lub aktualizuje właściwości istniejącego widoku, dostosowując je do tych zdefiniowanych w instrukcji.
 - Obsługiwane są następujące modyfikacje:
 - Przekształcanie w widok zabezpieczony (lub przywracanie z widoku zabezpieczonego).
 - Dodawanie, nadpisywanie i usuwanie komentarza dla widoku lub kolumn widoku.
 - Włączanie lub wyłączanie śledzenia zmian dla widoku.

Widoki – opcjonalne parametry

- SECURE - Określa, że widok jest zabezpieczony.
 - W przypadku widoków niezabezpieczonych wewnętrzne optymalizacje mogą pośrednio ujawniać dane.
 - Niektóre wewnętrzne optymalizacje widoków wymagają dostępu do danych bazowych w tabelach, na których opiera się dany widok. Dostęp ten może spowodować, że dane ukryte przed użytkownikami widoku zostaną ujawnione za pośrednictwem kodu użytkownika, np. funkcji zdefiniowanych przez użytkownika lub innych metod programowych. Widoki zabezpieczone nie korzystają z tych optymalizacji, co gwarantuje, że użytkownicy nie mają dostępu do danych bazowych.

Widoki – opcjonalne parametry

- SECURE - Określa, że widok jest zabezpieczony.
 - W przypadku widoku niezabezpieczonego definicja widoku jest widoczna dla innych użytkowników.
 - Domyślnie wyrażenie zapytania używane do utworzenia widoku standardowego, zwane również definicją widoku lub tekstem, jest widoczne dla użytkowników w różnych poleceniach i interfejsach.
- Ze względów bezpieczeństwa lub ochrony prywatności użytkownik może nie chcieć ujawniać tabel bazowych ani szczegółów wewnętrznej struktury widoku. W przypadku widoków zabezpieczonych definicja widoku i jego szczegóły są widoczne wyłącznie dla uprawnionych użytkowników (tj. użytkowników, którym przyznano rolę właściciela widoku).

Widoki – opcjonalne parametry

- SECURE
 - Widoki należy uznać za zabezpieczone, gdy są one wyraźnie przeznaczone do ochrony prywatności danych (tj. w celu ograniczenia dostępu do danych wrażliwych, które nie powinny być udostępniane wszystkim użytkownikom tabel bazowych).
 - Widoków zabezpieczonych nie należy stosować w przypadku widoków zdefiniowanych wyłącznie dla wygody użytkowników, takich jak widoki tworzone w celu uproszczenia zapytań, w przypadku których użytkownicy nie muszą rozumieć sposobu reprezentacji danych bazowych. **Widoki zabezpieczone mogą działać wolniej niż widoki niezabezpieczone.**

Widoki – opcjonalne parametry

- **SECURE**
 - Definicja widoku zabezpieczonego jest widoczna wyłącznie dla uprawnionych użytkowników (tj. użytkowników, którym przyznano rolę będącą właścicielem tego widoku). Jeśli nieuprawniony użytkownik użyje któregośkolwiek z poniższych poleceń lub interfejsów, definicja widoku nie zostanie wyświetlona:
 - **SHOW VIEWS**
 - **SHOW MATERIALIZED VIEWS**
 - **GET_DDL**
 - **VIEWS**

Widoki – opcjonalne parametry

- SECURE
 - W przypadku widoków niematerialnych kolumna IS_SECURE w widokach schematu informacyjnego i widokach wykorzystania konta wskazuje, czy dany widok jest zabezpieczony. Na przykład w przypadku widoku o nazwie MYVIEW w bazie danych mydb:

```
select table_catalog, table_schema, table_name, is_secure
from mydb.information_schema.views
where table_name = 'MYVIEW';
```

```
select table_catalog, table_schema, table_name, is_secure
from snowflake.account_usage.views
where table_name = 'MYVIEW';
```


Widoki – opcjonalne parametry

- **SECURE**

- Uwzględnienie uprawnień można zintegrować z użytkownikami i rolami w Snowflake za pomocą funkcji kontekstowych `CURRENT_ROLE` i `CURRENT_USER`. Poniższy przykład ilustruje wykorzystanie ról do kontrolowania dostępu do wierszy tabeli. Oprócz tabeli zawierającej dane (widgets) w przykładzie wykorzystano tabelę dostępu (widget_access_rules) w celu śledzenia, które role mają dostęp do poszczególnych wierszy w tabeli danych:

```
CREATE TABLE widgets (  
  id NUMBER(38,0) DEFAULT widget_id_sequence.nextval,  
  name VARCHAR,  
  color VARCHAR,  
  price NUMBER(38,0),  
  created_on TIMESTAMP_LTZ(9));  
CREATE TABLE widget_access_rules (  
  widget_id NUMBER(38,0),  
  role_name VARCHAR);  
CREATE OR REPLACE SECURE VIEW widgets_view AS  
  SELECT w.*  
    FROM widgets AS w  
   WHERE w.id IN (SELECT widget_id  
                  FROM widget_access_rules AS a  
                  WHERE upper(role_name) = CURRENT_ROLE()  
                  )  
;
```

Widoki – opcjonalne parametry

- `{ [ { LOCAL | GLOBAL } ] TEMP | TEMPORARY | VOLATILE }`
 - Określa, że widok istnieje tylko przez czas trwania sesji, w której został utworzony. Widok tymczasowy wraz z całą zawartością zostaje usunięty po zakończeniu sesji.
 - Synonimy i skróty dla TEMPORARY (np. GLOBAL TEMPORARY) są udostępniane w celu zapewnienia zgodności z innymi bazami danych (np. w celu zapobiegania błędom podczas migracji instrukcji CREATE VIEW). Widoki utworzone przy użyciu dowolnego z tych słów kluczowych wyglądają i działają identycznie jak widoki utworzone przy użyciu słowa kluczowego TEMPORARY.
 - Domyślnie: Brak wartości. Jeśli widok nie jest zadeklarowany jako TEMPORARY, jest on stały.
 - Jeśli utworzono widok tymczasowy o tej samej nazwie co inny widok w schemacie, wszystkie zapytania i operacje wykonywane na tym widoku mają wpływ tylko na widok tymczasowy w sesji, dopóki nie usunie się widoku tymczasowego. Usunięcie widoku powoduje usunięcie widoku tymczasowego, a nie widoku już istniejącego w schemacie.

Widoki – opcjonalne parametry

- **RECURSIVE**

- Określa, że widok może odwoływać się do samego siebie przy użyciu składni rekurencyjnej bez konieczności stosowania wyrażenia CTE (common table expression).
- Wartość domyślna: brak wartości (widok nie jest rekurencyjny lub jest rekurencyjny wyłącznie przy użyciu wyrażenia CTE)
- Przykład: <https://docs.snowflake.com/en/user-guide/views-introduction#label-views-recursive>

Widoki – opcjonalne parametry

- **MASKING POLICY = *policy_name***
 - Określa zasady maskowania, które mają zostać zastosowane do kolumny.
 - Więcej informacji: <https://docs.snowflake.com/en/user-guide/security-column-intro>
- **PROJECTION POLICY *policy_name***
 - Określa zasady rzutowania, które mają zostać zastosowane do kolumny.
 - Więcej informacji: <https://docs.snowflake.com/en/user-guide/projection-policies>

Widoki – opcjonalne parametry

- **CHANGE_TRACKING = { TRUE | FALSE }**
 - Określa, czy w widoku ma być włączone śledzenie zmian.
 - Wartość TRUE włącza śledzenie zmian w widoku. To ustawienie dodaje parę ukrytych kolumn do tabeli źródłowej i rozpoczyna przechowywanie metadanych śledzenia zmian w tych kolumnach. Kolumny te zajmują niewielką ilość miejsca.
 - Metadane śledzenia zmian można pobrać za pomocą klauzuli CHANGES w instrukcjach SELECT lub poprzez utworzenie i zapytanie jednego lub więcej strumieni w tabeli.
 - Wartość FALSE nie włącza śledzenia zmian w widoku.

Widoki – opcjonalne parametry

- **ROW ACCESS POLICY** *policy_name* ON (*col_name* [, *col_name* ...])
 - Określa zasady dostępu do wierszy, które mają zostać ustawione dla widoku.
 - Snowflake obsługuje zabezpieczenia na poziomie wierszy poprzez stosowanie zasad dostępu do wierszy, które określają, które wiersze mają zostać zwrócone w wyniku zapytania.
 - Zasada dostępu do wierszy może być stosunkowo prosta i zezwalać jednej konkretnej roli na przeglądanie wierszy, lub bardziej złożona i zawierać tabelę mapowania w definicji zasady, która określa dostęp do wierszy w wyniku zapytania.
 - Więcej informacji: <https://docs.snowflake.com/en/user-guide/security-row-intro>

Widoki zmaterializowane

- **CREATE MATERIALIZED VIEW**

- Tworzy nowy widok zmaterializowany w bieżącym lub wskazanym schemacie na podstawie zapytania do istniejącej tabeli i wypełnia ten widok danymi.
- Utworzenie widoku zmaterializowanego wymaga uprawnień CREATE MATERIALIZED VIEW w schemacie oraz uprawnień SELECT do tabeli bazowej.
- Jeśli w definicji widoku zostanie określona funkcja CURRENT_DATABASE lub CURRENT_SCHEMA, funkcja ta zwraca bazę danych lub schemat zawierający widok, a nie bazę danych lub schemat używany w sesji.
- Wybierając nazwę widoku zmaterializowanego, należy pamiętać, że schemat nie może zawierać tabeli i widoku o tej samej nazwie. Polecenie CREATE [MATERIALIZED] VIEW generuje błąd, jeśli w schemacie istnieje już tabela o tej samej nazwie.

Widoki zmaterializowane

- **MATERIALIZED VIEW**

- Podczas definiowania instrukcji `select_statement` należy pamiętać o następujących kwestiach:

- Nie można używać klauzuli `HAVING` ani `ORDER BY`.
- Jeśli w widoku zmaterializowanym zastosowano klauzulę `CLUSTER BY`, konieczne jest użycie klauzuli `column_list`.
- Jeśli w instrukcji `select_statement` odwołuje się do tabeli bazowej więcej niż raz, należy używać tego samego kwalifikatora dla wszystkich odwołań do tabeli bazowej.
- Na przykład nie należy używać mieszanki form `base_table`, `schema.base_table` i `database.schema.base_table` w tej samej instrukcji `select_statement`. Zamiast tego należy wybrać jedną z tych form (np. `database.schema.base_table`) i stosować ją konsekwentnie w całej instrukcji `select_statement`.
- Nie należy wysyłać zapytań do obiektów strumieniowych w instrukcji `SELECT`. Strumienie nie są przeznaczone do pełnienia roli obiektów źródłowych dla widoków lub widoków zmaterializowanych.

Widoki zmaterializowane

- **MATERIALIZED VIEW**

- Podczas definiowania instrukcji `select_statement` należy pamiętać o następujących kwestiach:

- Nie można używać klauzuli `HAVING` ani `ORDER BY`.
- Jeśli w widoku zmaterializowanym zastosowano klauzulę `CLUSTER BY`, konieczne jest użycie klauzuli `column_list`.
- Jeśli w instrukcji `select_statement` odwołuje się do tabeli bazowej więcej niż raz, należy używać tego samego kwalifikatora dla wszystkich odwołań do tabeli bazowej.
- Na przykład nie należy używać mieszanki form `base_table`, `schema.base_table` i `database.schema.base_table` w tej samej instrukcji `select_statement`. Zamiast tego należy wybrać jedną z tych form (np. `database.schema.base_table`) i stosować ją konsekwentnie w całej instrukcji `select_statement`.
- Nie należy wysyłać zapytań do obiektów strumieniowych w instrukcji `SELECT`. Strumienie nie są przeznaczone do pełnienia roli obiektów źródłowych dla widoków lub widoków zmaterializowanych.

Dynamiczne tabele

- **DYNAMIC TABLE**

- Tabela dynamiczna materializuje wyniki zapytania SELECT i na bieżąco je aktualizuje.
- Wystarczy podać zapytanie SELECT oraz docelowy okres opóźnienia, a Snowflake śledzi zależności i odświeża dane zgodnie z harmonogramem.
- Instrukcja CREATE określa zawartość tabeli (zapytanie SELECT), wymagany poziom aktualności danych (TARGET_LAG) oraz sposób odświeżania danych przez Snowflake (REFRESH_MODE).
- Po utworzeniu tabeli Snowflake monitoruje tabelę bazową i automatycznie odświeża tabelę dynamiczną.

Dynamiczne tabele

- **W jaki sposób Snowflake dba o aktualność tabel dynamicznych**
 - Podczas tworzenia tabeli dynamicznej Snowflake analizuje zapytanie, identyfikuje tabele bazowe, z których pobiera dane, i rejestruje tabelę dynamiczną w celu jej odświeżania.
 - Od tego momentu Snowflake monitoruje tabele bazowe pod kątem zmian i odświeża tabelę dynamiczną, aby utrzymać ją w ramach ustawionego docelowego opóźnienia.
 - Snowflake wykrywa, że tabele bazowe uległy zmianie.
 - Jeśli tabela dynamiczna korzysta z odświeżania przyrostowego, obliczane są tylko te wiersze, które uległy zmianie. Jeśli korzysta z odświeżania pełnego, odświeżany jest cały zbiór wyników.
 - Nowe wyniki są stosowane do tabeli dynamicznej w sposób atomowy, dzięki czemu użytkownicy nigdy nie widzą częściowego odświeżenia.

Dynamiczne tabele

- **W jaki sposób Snowflake dba o aktualność tabel dynamicznych**
 - W przypadku wielu tabel dynamicznych, które są od siebie zależne, Snowflake traktuje je jako potok.
 - Na podstawie zapytań ustala schemat zależności i wybiera spójny znacznik czasu migawki.
 - Snowflake odświeża tabele zgodnie z kolejnością zależności, dzięki czemu tabele znajdujące się dalej w potoku zawsze widzą spójny obraz danych wejściowych z tabeli poprzedzających.

Dynamiczne tabele

- **Koszt tabel dynamicznych**
 - Każda dynamiczna tabela generuje koszty w trzech kategoriach. Waga względna każdej z nich zależy od częstotliwości odświeżania, wielkości hurtowni danych oraz ilości danych.

Komponent	Czynniki wpływające na koszt	Sposób rozliczenia
Virtual warehouse compute	Odśwież zapytania uruchamiane w przypisanej bazie danych. Większa baza danych lub częstsze odświeżanie powodują wzrost kosztów.	Kredyty przyznawane co sekundę w zależności od wielkości magazynu
Cloud Services compute	Kontrole metadanych, które wykrywają zmiany w źródłach danych i określają, czy konieczne jest odświeżenie hurtowni danych. Działa w każdym cyklu odświeżania, nawet jeśli dane nie uległy zmianie.	Rozliczane codziennie, ale tylko wtedy, gdy środki na koncie usług w chmurze przekroczą 10% całkowitej kwoty środków na koncie magazynowym z danego dnia
Storage	Wyniki w postaci danych, dane dotyczące podróży w czasie oraz kopie zapasowe. Odświeżanie przyrostowe powoduje dodanie niewielkiej kolumny metadanych dla każdego wiersza.	Standardowe stawki za przechowywanie danych w Snowflake. Dynamiczne tabele Apache Iceberg korzystają z pamięci zewnętrznej i nie wiążą się z kosztami przechowywania danych w Snowflake.

Dynamiczne tabele

- **Jeżeli...**
 - Jeśli potrzebujesz deklaratywnego potoku SQL, skorzystaj z tabel dynamicznych
 - Jeśli potrzebujesz logiki proceduralnej lub niestandardowej koordynacji, skorzystaj ze strumieni i zadań (task and streams)
 - Jeśli potrzebujesz przyspieszyć wykonywanie zapytań dotyczących pojedynczego tabeli, skorzystaj z widoków zmaterializowanych

Dynamiczne tabele

- **Kiedy nie używać?**
 - Potrzebujesz poleceń DML – dynamic table są read-only
 - Potrzebujesz procedur i zewnętrznych funkcji
 - Potrzebujesz odświeżenia częściej niż co 1 minutę
 - Chcesz używać MERGE

```
CREATE DYNAMIC TABLE my_dynamic_table (pre_tax_profit, taxes, after_tax_profit)
  TARGET_LAG = '20 minutes'
  WAREHOUSE = mywh
  AS
  SELECT revenue - cost, (revenue - cost) * tax_rate, (revenue - cost) * (1.0 - tax_rate)
  FROM staging_table;
```

Tabela porównawcza

Cecha	View	Materialized View	Dynamic Table
Cel główny	Abstrakcja logiczna i governance	Przyspieszenie odczytu przez precomputation	Deklaratywny pipeline transformacyjny
Przechowywanie danych	Nie, wynik logiczny	Tak, wynik precomputed	Tak, wynik materializowany
Aktualność	Zależna od zapytania do źródeł	Utrzymywana przez Snowflake	Utrzymywana wg target lag / refresh mode
Koszt	Głównie przy odczycie	Storage + maintenance compute	Refresh compute + storage
Najlepszy use case	Semantic layer, secure access	Częste, drogie query na dużych tabelach	ETL/ELT i wieloetapowe transformacje